

Internet Technology Lab Report

Assignment – 3

Problem Statement:

Implement a WebSocket-based Photo Sharing Application. The application must be an asynchronous multi-client system where users can share photos and comments publicly or within specific groups using a Node.js framework

Student Details

- **Name:** Sumesh Ranjan Majee
 - **Roll Number:** 002310501020
 - **Year:** 3rd Year B.C.S.E
 - **Group:** A1
 - **Date:** 18th February 2026
-

Subject:

Internet Technology

1. Design

1.1 Purpose

The purpose of this program is to implement a Full-Duplex communication system using the WebSocket protocol. Unlike traditional HTTP (which follows a Request-Response model), this application maintains a persistent connection between the client and server. This allows for:

1. **Real-time Broadcasting:** Instant updates to all users (Global Feed).
2. **Multicasting:** Targeted delivery of messages/images to specific subsets of users (Group/Room logic).
3. **Asynchronous I/O:** Handling multiple file uploads and chat messages concurrently without blocking the server.

1.2 System Architecture

The application follows a standard Client-Server architecture utilizing **Node.js** and **Socket.io**.

- **Server Side:** Acts as the central hub. It stores message history in memory (globalHistory array and groupHistory object) and routes messages. It listens on HTTP Port 3000 and upgrades connections to WebSockets.
- **Client Side:** A browser-based interface. It manages local state (current room, username) and caches history using the browser's localStorage for resilience.

Procedural Organization (Data Flow):

1. **Connection:** Client initiates handshake $\rightarrow$ Server accepts $\rightarrow$ Server sends cached history.
2. **Public Event:** Client emits public_message $\rightarrow$ Server saves to globalHistory $\rightarrow$ Server broadcasts via io.emit.
3. **Group Event:** Client emits join_group $\rightarrow$ Server associates socket with Room ID $\rightarrow$ Server routes future messages via io.to(room).emit.

1.3 Input and Output Format

- **Input:**
 - **Text:** UTF-8 strings for usernames, room names, and chat messages.
 - **Images:** Binary image files selected via `<input type="file">`. These are converted into **Base64** encoded strings on the client side before transmission to ensure they can be

embedded in JSON payloads.

- **Output:**

- **JSON Payloads:** The server responds with structured objects, e.g., { id: "...", text: "Hello", image: "data:image/png;base64...", username: "Sumesh" }.
- **Visual Representation:** The client renders these payloads as chat bubbles containing text and elements.

2. Implementation

The application is built using the **Express** framework for serving static files and **Socket.io** for handling WebSocket events.

2.1 Key Method Descriptions (Server)

1. Group Management (Room Logic)

The server uses Socket.io's "Rooms" feature to segregate traffic.

JavaScript

```
// server.js
socket.on("join_group", (data) => {
  const { room, username } = data;
  // ... validation logic ...
  socket.join(room); // Adds the socket to a specific channel
  socket.emit("history_response", groupHistory[room]); // Send room-specific history
  socket.to(room).emit("system_message", `${username} has joined...`);
});
```

Rationale: This ensures privacy and efficiency. Messages sent to "Group A" are never transmitted to clients in "Group B".

2. Message Broadcasting & Storage

We implement a "Ring Buffer" strategy (max 50 messages) to prevent memory leaks.

JavaScript

```
// server.js
socket.on("public_message", (data) => {
  globalHistory.push(data);
  if (globalHistory.length > 50) globalHistory.shift(); // Maintain fixed size
  io.emit("receive_public", data); // Broadcast to ALL connected clients
});
```

2.2 Key Method Descriptions (Client)

1. Base64 Image Conversion

Since WebSockets handle text efficiently, we convert images to data strings.

JavaScript

```
// script.js
function getBase64(file) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.readAsDataURL(file); // Converts binary -> Base64 string
    reader.onload = () => resolve(reader.result);
    // ... error handling
  });
}
```

3. Test Cases

To verify the correctness of the program, the following test cases were executed.

Test Case ID	Test Description	Sample Data / Action	Expected Result	Actual Result
TC-01	Global Broadcast	User A sends "Hello World" in Global Mode.	User B (in a different tab) receives "Hello World"	Pass

			instantly.	
TC-02	Image Sharing	User uploads photo.jpg.	The image renders as an tag on all connected clients.	Pass
TC-03	Group Isolation	User A joins "Room1". User B joins "Room2". User A sends a message.	User B does not receive the message.	Pass
TC-04	History Caching	User refreshes the page.	Previous messages reload from localStorage or Server History.	Pass
TC-05	Edit Message	User A edits a sent message.	The text updates on all screens with an "(edited)" tag.	Pass
TC-06	Delete Message	User A deletes a message.	The message div is removed from the DOM for all users.	Pass

4. Results

Performance Metrics:

- **Latency:** Messages are delivered in < 50ms on a local network.
- **Concurrency:** The server successfully handled multiple concurrent connections (tested with 3 different browsers).
- **Data Handling:** The maxHttpBufferSize: 1e8 (100MB) configuration in server.js allowed

for high-resolution images to be shared without crashing the socket connection.

Visual Output:

The application successfully renders a split interface:

1. **Global Feed:** Shows messages from all users.
2. **Group Chat:** Activated only when a user creates/joins a room, showing a green system status bar.

5. Analysis

5.1 Discussion on Results

The implementation successfully meets the assignment requirements for asynchronous photo sharing. The use of **Socket.io** abstracted the complexity of handshake management and keeping the connection alive.

Protocol Layers Created:

1. **Application Layer:** JSON-based custom protocol (e.g., event types `public_message`, `edit_message`).
2. **Session Layer:** Managed by Socket.io (Heartbeats/Pings to detect disconnection).
3. **Transport Layer:** TCP (underlying WebSocket connection).

5.2 Correctness and Error Handling

- **Input Validation:** The client prevents sending empty messages (unless an image is attached).
- **Room Validation:** The server prevents joining non-existent groups (`group_error` event).
- **State Consistency:** The `edit_message` and `delete_message` events ensure that the view is consistent across all clients, not just the sender.

5.3 Known Limitations & Improvements

1. **Volatile Storage:** All server-side history is stored in variables (`globalHistory`). If the server restarts, history is lost.
 - *Improvement:* Integrate a database like MongoDB.
 2. **Base64 Overhead:** Converting images to Base64 increases file size by ~33%.
 - *Improvement:* Upload images to a cloud bucket (AWS S3) and share only the URL.
-

6. Comments

Evaluation:

This assignment was highly educational. It demonstrated the shift from "stateless" HTTP web development to "stateful" real-time programming.

Learning Outcomes:

1. **Event-Driven Programming:** I learned how to handle asynchronous events (`socket.on`) rather than linear procedural code.
2. **Data Serialization:** Understanding how to package binary data (images) for transmission over text-based protocols.
3. **Room/Namespace Concepts:** How to logically separate users on a single physical server port.

Difficulty:

The assignment was of **moderate difficulty**. The most challenging part was synchronizing the "Edit" and "Delete" actions across all clients, ensuring the specific DOM element (`msg-{id}`) was targeted correctly in `script.js`.
